

From Packing to Routing: Cross-Layer Design Space Exploration for IMC DNN Accelerators on 3D NoCs

Abstract—In-memory computing removes weight traffic from DNN accelerators but relocates the rest onto the on-chip network, where hundreds of crossbar tiles reduce partial sums through a few accumulator ports. Communication is then set by three decisions — how crossbars are packed into tiles, where tiles are placed, and how packets are routed — each optimised today with the others held fixed. This paper explores the three as one design space on a 3D NoC, in cycle-accurate simulation of three DNN blocks. The layers couple through one quantity, the port floor that packing fixes: choosing packing for that floor rather than for traffic volume lowers mean delay $11.6\times$ and p99 $18.6\times$; below the floor, a zero-cost placement refinement recovers up to 46% of delay and 67% of p99; and adaptive selection wins in both regimes ($1.85\times/2.39\times$ above, $1.15\times/1.23\times$ below) yet leaves a residue below the floor that no offline metric predicts. Packing thus sets the operating point that placement can only reach and routing can only defend, while the residue routing leaves is temporal, calling for selection that adapts in time as well as space.

I. INTRODUCTION

Deep Neural Networks (DNNs) have become the dominant workload class across computing. They underpin the vast majority of AI-based applications, and DNN inference has become infrastructure, running continuously from edge devices to warehouse-scale data centers [1]. The latency and energy budget of DNN accelerators is therefore a first-order design metric, and a primary axis of competition among chip designers and manufacturers. Weight movement is what drives this budget in conventional accelerators [2], [3]. In-Memory Computing (IMC) eliminates weight traffic by keeping weights in place and performing Multiply-Accumulate (MAC) operations locally. This alleviates the weight-movement bottleneck, improving performance while reducing the communication energy budget and launching a new generation of IMC DNN accelerators [4], [5], [6]. However, no single array capacity can occupy all weights for a full DNN, making tiled architecture inevitable. For example one ResNet-50 block requires 736 crossbars (CBs) and a full network requires over ten thousand. Because ADC peripherals have a large area and power overhead, they are shared across the area of a tile. This limits the number of CBs that fit in a tile resulting in the computation being split among hundreds or even thousands of tiles connected through a Network-on-Chip (NoC) [5].

The consequence is that IMC does not eliminate communication; it relocates it. What remains mobile is activation scatter traffic and the dominant partial-sum reduction traffic: with default packing, roughly 82% of a block's NoC bytes

is reduction that converge on few accumulator tiles via the on-chip network. The accelerator's bottleneck that governs its performance and energy becomes communication, and the design focus shifts from being computation-centric to communication-centric. Communication on IMC DNN accelerators is not set by a single design choice, nor within a single design layer, but by a chain of dependent choices taken at different layers.

This paper presents a cross-layer design space exploration spanning three main layers—packing, mapping and routing—in which the design choices available at each layer are evaluated both for their performance impact within that layer and for their impact on the layers downstream. A delay- and inference-time-aware design flow is proposed, and is shown to significantly reduce both average and tail delays, the latter measured as the p99 latency that governs inference time. The contributions are: The contributions are:

- a cross-layer design-space exploration for IMC-based DNN accelerators, spanning the packing, mapping and routing layers, in which each layer is evaluated for its own effect and for the constraints it imposes downstream;
- an attribution of communication delay to the layer that controls it: an invariant port floor fixed by the packing orientation and a peak link load fixed by the placement, with the binding term determining which layer holds control;
- three conditional couplings between adjacent layers, including a reversal in the sign of the routing policy's benefit under a change of placement alone;
- a design flow that follows from the attribution, together with a mapping-layer design-time objective that improves delay at the knee at no communication-cost overhead;
- a bounding result: no offline quantity, over an exhaustive search, predicts which placements benefit from adaptive selection, which establishes the limit of design-time optimisation and motivates online adaptivity.

Results show that the achieved improvement over the baseline reaches up to $11.6\times$ in mean delay and up to $18.6\times$ in p99 at matched load, at the packing layer; up to 46% in mean delay and 67% in p99 with zero communication-cost overhead, at the mapping layer; and up to $1.85\times$ in mean delay and $2.39\times$ in p99 over a local heuristic, at the routing and selection layer. Each gain was measured with the upstream layers already at their best, so downstream layers add to the upstream ones

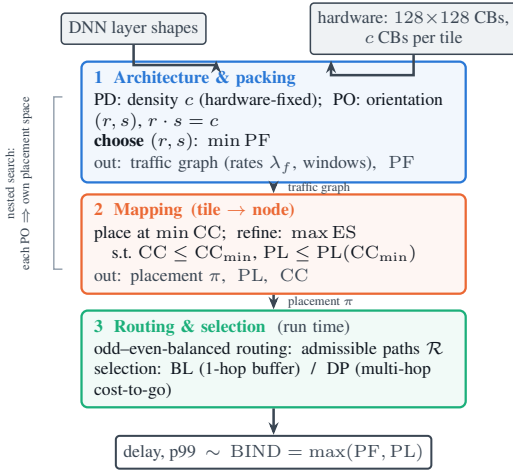


Fig. 1. The layered design flow: what each layer fixes, what it chooses, and what it passes down. Each step is evaluated with the layers above it at their chosen point.

rather than trading against them.

II. BACKGROUND AND RELATED WORK

Communication is a recognised bottleneck of DNN accelerators. Dataflow accelerators attack it by scheduling data reuse so that fewer bytes move [7], [3]; IMC accelerators attack it by keeping weights stationary in crossbar tiles [5], [6], [8], which removes weight traffic but relocates the rest onto the on-chip network, making the interconnect the design object [9].

The closest precedent is Krishnan et al. [10], who map DNN layers to tiles on a mesh and drive the NoC with per-pair injection rates, the traffic model adopted here, and conclude that the network is uncongested at average rates — a conclusion about the mean that Sec. IV-E qualifies. The same group models IMC and its NoC end to end [11] and synthesises a NoC per DNN [12], taking topology as the variable where this work keeps a regular 3D mesh and varies routing and selection. On 3D NoCs, TTNM [13] and TEFLON [14] optimise placement or fabric with the packing given; adaptive routing and selection [15], [16], [17] have been developed largely on synthetic traffic. At wafer scale the Cerebras WSE provisions more fabric than memory bandwidth [18]: past a certain integration the problem is communication-centric by construction.

Each of these works optimises one layer with the others held fixed: packing for utilisation or traffic volume, placement for hops or heat, routing for a given placement. None asks how a choice at one layer changes what the next can achieve. The results below show that this is where the largest effects lie: the minimum-traffic packing is the wrong one, placement matters only in a regime that packing decides, and what selection yields depends on both.

III. FORMULATIONS AND METHODOLOGY

Classical NoC traffic differs dramatically from DNN traffic on a tiled IMC substrate. Most of a block’s NoC traffic is partial-sum reduction, sunk by a few accumulator nodes,

so that a handful of ejection ports become the hotspots of the whole network. This makes *DNN traffic a funnel, not a permutation*. Thus the binding bottleneck is a port and not a link, and it is unaffected by mapping and routing unless mapping or routing creates a hotspot link whose load exceeds the port bottleneck — an extreme case, as this study shows and intuition supports (a hot accumulator has one ejection port but up to six incoming links).

Based on the above, the *traffic graph itself becomes a design variable, not an input*. Packing determines how much of the reduction is internalized on-tile and how much reaches the NoC, before any mapping or routing exists. Temporally, the traffic is non-stationary and bursty — layer-sequential execution gives each flow a disjoint activity window inside one inference period, so peak and average load diverge: the average decides whether a packing is feasible, while the peak decides the delay it will incur. Congestion is therefore transient and clustered at phase boundaries, on the timescale an adaptive policy’s cost field need track if it is to work. Finally, the endpoint that matters is the tail rather than the mean, because reduction is a barrier in which the accumulator waits for the slowest arriving partial sum — the arrival that determines the inference time.

The rest of this section defines the quantities these properties demand: the three design layers, the design-time quantities that determine each layer’s decision, the port and link load characterizations, and the proposed optimization objectives. It also describes the search that turns these into the proposed design flow.

A. The Three Design Layers

This work proposes a cross-layer design flow decomposed into three layers, in which the design decisions and constraints made at a layer are inherited by the layers below it. Every layer is therefore evaluated not only for its own performance but also for its downstream effect.

The top layer is *architecture and packing*. For a given hardware design it fixes how many crossbars share a tile (packing density, PD), and it decides the *layer-to-tile* mapping, i.e. how a DNN layer’s rows (reduce, r) and columns (scatter, s) are grouped across those crossbars (packing orientation, PO). Together, PD and PO fix how much of a layer’s traffic has to pass through a single tile port — a floor on delay that no later decision can lower. The second layer is *tile-to-node* placement, or *mapping*, which assigns NoC mesh nodes to those tiles and sets the peak single-link load across the NoC (PL). The third is the *routing and selection* policy, which decides, at run time, how much of that load can be steered off that link.

The layers are searched in that order, and the search is nested, not sequential. Each PO creates its own traffic graph and its own mapping space, so two POs can be compared only when both are at their best mappings. Comparing a PO with a tuned mapping against a PO with an arbitrary one confounds the two layers (packing and mapping).

B. Packing Density, Orientation and Port Floor

A packing (c, r, s) determines how crossbars of a DNN layer are split among tiles, with $c = r \cdot s$, where r defines reduction dimension grouping, internalizing partial-sum traffic on-tile, and s defines scatter dimension grouping, internalizing input activation broadcast. Packing results in a *traffic graph* whose nodes are logical tiles and arcs are inter-tile data flows. Flow f is characterized by data rate λ_f flits/cycle from tile u_f to tile v_f and is active over the window $[t_f^{\text{on}}, t_f^{\text{off}})$ of the inference period T .

A tile's injection (outbound) and ejection (inbound) channels are different which causes tile's two port rates bind separately. The elementary intervals $\mathcal{I}$ are the maximal windows on which the active flow set is constant. Following the above formulation Peak injection (PIL) and ejection (PEL) loads and the *port floor* (PF) are

$$\text{PIL} = \max_{I \in \mathcal{I}, n \in \mathcal{N}} \sum_{f: u_f=n, f \text{ active in } I} \lambda_f, \quad (1)$$

$$\text{PEL} = \max_{I \in \mathcal{I}, n \in \mathcal{N}} \sum_{f: v_f=n, f \text{ active in } I} \lambda_f, \quad (2)$$

$$\text{PF} = \max(\text{PIL}, \text{PEL}). \quad (3)$$

where I ranges over $\mathcal{I}$ and n over the tiles of the traffic graph $(\mathcal{N})$.

These metrics are mapping-independent because equations (1)–(3) involve no placement variable, so PIL, PEL and hence PF are *placement-invariant*: remapping determines which NoC node is hot, but not how hot it is. PF is the quantity that the packing layer controls. The upper bound on the load the system can sustain in a burst is $1/\text{PF}$.

C. Mapping: Peak Link Load

A mapping is an injective map π that assigns tiles to mesh nodes. Once π is fixed, the inter-tile flows are carried on NoC links through the routing function $\mathcal{R}$. Let $\varphi_f(\ell, \pi) \in [0, 1]$ be the fraction of flow f 's minimal $\mathcal{R}$ -admissible paths that traverse directed link ℓ . An even spread over admissible paths gives the offered link load $\Lambda_\ell = \sum_f \lambda_f \varphi_f(\ell, \pi)$, again maximized over $\mathcal{I}$. The mapping layer is then characterized by peak link load (PL) and communication cost (CC) — a hop-count energy proxy

$$\text{PL} = \max_{\ell} \Lambda_\ell, \quad (4)$$

$$\text{CC} = \sum_f \lambda_f h(\pi(u_f), \pi(v_f)). \quad (5)$$

with h the Manhattan hop distance. PL and CC are determined by π ; PF is fixed at the upstream packing layer. The link attaining PL is denoted ℓ^* .

The packing and mapping layers meet at the *binding load*

$$B = \max(\text{PF}, \text{PL}), \quad (6)$$

which is a design objective optimized by the proposed flow.

D. Routing and Selection

The third layer is *routing and selection*. It decides how traffic is relayed for a given packing and mapping. Routing computes the set of allowed paths $\mathcal{R}$, which binds to the mapping layer through $\varphi_f(\ell, \pi)$. More permissive paths for a flow f help to spread it over more links, lowering PL for the same π .

Selection then determines which output, among the admissible ones, a packet is forwarded to. It is the only decision that can respond to congestion (or any run-time optimization metric) rather than predict it. However, it is bounded by what the upstream layers have left: load can be moved off ℓ^* only insofar as the flows sharing it have alternatives. Thus, mapping has an impact on what selection can achieve, provided a proper offline metric captures the escapable load. This is the key idea behind the metrics proposed in this work and described below.

E. Proposed Metrics: Escapable Load Fraction and Slope

The proposed metrics stem from the fact that *selection can only divert load that has somewhere to go*. A flow whose fraction on link ℓ is 1 ($\varphi_f(\ell) = 1$) is *forced* through ℓ . We define the *escapable fraction* for link ℓ as the share of its load carried by flows with an alternative path

$$E(\ell) = 1 - \frac{1}{\Lambda_\ell} \sum_{f: \varphi_f(\ell)=1} \lambda_f, \quad (7)$$

i.e. it is the complement of the *forced* load. We write escapable fraction of the link with peak load (PL) $E = E(\ell^*)$.

The hottest link escape, however, does not fully account for how the escape room is distributed across the congested links.

Let $\mathcal{L}_q$ be the top- q fraction of links ordered by Λ_ℓ , and let $E_q = \sum_{\ell \in \mathcal{L}_q} E(\ell) \Lambda_\ell / \sum_{\ell \in \mathcal{L}_q} \Lambda_\ell$ be the load-weighted escapable fraction of that tier. We propose the *escape slope*

$$\text{ES} = E_{q_2} - E_{q_1}, \quad (8)$$

the change in escape room between the top q_1 and the top q_2 of links ($q_2 > q_1$; we use $q_1 = 5\%$, $q_2 = 20\%$). ES is computed offline from the logical traffic graph and the mapping alone.

ES stems from the observation that hot links are typically created by forced traffic, whose load cannot move, while the load queued on the links feeding them can. Thus, high ES means high escape room just below the peak, allowing backed-up load to drain sideways before it reaches the forced high-load core (the top q_1). Conversely, low ES means the shoulder is as rigid as the core and nothing drains. To improve the performance of the selection policy, ES is therefore maximized.

IV. RESULTS

A. Experimental Tools and Setup

To evaluate the proposed design flow and metrics, we employ Noxim, a cycle-accurate SystemC-based NoC simulator [19]. Noxim is a 2D NoC simulator, so we extended it to enable 3D and implemented the routing and selection used in this work which were not available in the original version: namely the balanced odd-even routing [15] and dynamic programming network selection [16].

TABLE I

WORKLOADS, ASSUMING 128×128 CROSSBARS (CB). THE THREE BLOCKS SPAN A RESIDUAL CNN (RESNET-50), A PLAIN CNN (VGG-16) AND A TRANSFORMER ENCODER (DEiT-S).

workload	block	layers	weights (M / CBs)	period (kcyce)	act. (MB)	psum (MB)
ResNet-50	stage-3	4 (3 conv + shortcut)	1.51 / 736	38.5	0.80	9.2
DeiT-S	encoder	6 (q,k,v,o,ff1,ff2)	1.77 / 864	58.1	1.36	10.9
VGG-16	block 3	3 (all conv)	1.47 / 720	115.6	4.42	144.5

The DP cost field reconvergence time is determined by the topology diameter and node count. For the target mesh it is 648 cycles with DP clocked at $4 \times$ the network clock.

The 3D NoC size is a $6 \times 6 \times 3$ mesh of 108 tiles. Each tile holds c 128×128 crossbars of single-bit cells, with 8-bit weights spread over eight bit-planes. Flit size is 4 bytes, the packet is fixed at 16 flits, input buffers size is 16 flits, and the substrate runs at 1GHz clock. NoC links and tile's injection/ejection ports carry one 4-byte flit per 1 ns cycle, giving 4 GB/s per channel, with independent injection and ejection channels.

Changing the mesh simultaneously redefines both the mapping space and the routing substrate, because topology is not another layer below routing but a parallel design axis. Thus, in this work the topology is fixed to keep the layer attribution identifiable.

B. Workloads and Traffic Generation

Workloads are ResNet-50 [20], VGG-16 [21] and DeiT-S [22]. The three DNN workloads span a residual CNN, a plain CNN and a transformer encoder. They differ in partial-sum-to-activation ratio from $8 \times$ (DeiT-S) to $33 \times$ (VGG-16), and NoC bandwidth demand, by 5 – $6 \times$, and in structure: a deep reduction funnel with a parallel shortcut (ResNet-50), load distributed across many sinks of similar weight with no isolated funnel (DeiT-S), and a sequence of deep reductions (VGG-16). They are converted offline into statistical traffic tables the simulator can run, with per-pair injection rates and phase windows. Table I lists the three workloads. A workload inference period (compute time) is identical across its packings, and the activation and partial-sum volumes shown are per inference and packing-independent.

All runs use an appropriate warm-up and are averaged over 3 to 30 seeds, with offered load swept by a scalar multiplier k on all injection rates. Unless stated otherwise, comparisons are made inside the knee window, defined as the load range in which mean BL delay stays below an empirically set multiple ($30 \times$) of its free-flow value. Because mappings differ in their saturation loads, capacity is measured per mapping as k^* : the injection scale at which a mapping's delay reaches twice its own free-flow delay. The mapping search and the computation of design-time metrics (PF, PL, CC, ES) are done in Python from the traffic tables. Approximately 36 000 simulation runs were conducted to obtain the results below.

– HERE–

TABLE II

MIN-PF VS. MIN-TRAFFIC ORIENTATION PER DENSITY; LAST COLUMN: FLOOR GAINED / TRAFFIC COST OF SELECTING ON PF (“—”: SAME ORIENTATION). PF IN FLITS/CYCLE.

workload	c	min-PF		min-traffic		floor / traffic
		(r, s)	PF	(r, s)	PF	
ResNet-50	8	(2,4)	0.392	(2,4)	0.392	—
	16	(2,8)	0.258	(4,4)	0.717	$2.78 \times / 1.08 \times$
	32	(4,8)	0.327	(4,8)	0.327	—
DeiT-S	8	(1,8)	0.678	(1,8)	0.678	—
	16	(1,16)	0.905	(4,4)	2.849	$3.15 \times / 1.62 \times$
	32	(2,16)	0.773	(4,8)	1.384	$1.79 \times / 1.67 \times$
VGG-16	8	(4,2)	0.958	(4,2)	0.958	—
	16	(4,4)	0.943	(8,2)	1.194	$1.27 \times / 1.25 \times$
	32	(8,4)	0.760	(16,2)	1.953	$2.57 \times / 1.07 \times$

C. Design Flow and Search Procedure

The flow evaluated below selects the orientation that minimizes PF, places at minimum communication cost, refines by maximizing ES subject to $CC \leq CC_{\min}$ and $PL \leq PL(CC_{\min})$, minimizes PL where search budget remains, and applies adaptive selection; each step is evaluated with the layers above it at their chosen point. Placements are searched by a guarded hill-climb from a random permutation, 300 steps of 50 candidate moves (a transposition, or with probability 0.3 a relocation to an idle node), the constraint applied in the move filter and the best strictly improving move accepted. All searches are seeded and reproducible.

D. Packing Impact

The packing layer acts before any placement exists, so we first ask what it alone decides. For each workload we enumerate every feasible (c, r, s) — 34 points in all — and evaluate PF offline from the traffic graph. Within each density c we identify the orientation minimising total NoC traffic, the classical pre-network objective, and the one minimising PF.

Table II reports both. The two objectives coincide in four of the nine (workload, c) cells and select different orientations in the remaining five. Where they differ, ranking on PF lowers the floor by 1.27 – $3.15 \times$ at a cost of 1.07 – $1.67 \times$ in traffic; since the inference period is fixed per workload, that is also the ratio of average NoC bandwidth. Across ResNet-50's eleven feasible points PF spans 0.258 – 1.494 flits/cycle, a factor of 5.8 . Orientation therefore fixes the floor, and no later decision can lower it.

Whether a lower floor is worth its traffic overhead cannot be settled offline, so we simulate the widest disagreement on ResNet-50: $c=16$, the min-PF orientation (2, 8) against the min-traffic orientation (4, 4), a $2.78 \times$ lower floor for 8% more traffic. To keep the comparison at the packing layer, both arms receive minimum-communication-cost placements and are swept over the same load ladder, three placements per arm at three seeds per rung, under local selection (BL).

Fig. 2 shows both arms across the ladder: the min-traffic arm bends between $k = 0.7$ and 1.0 and is saturated by 1.6 , while the min-PF arm bends at 2.0 – 2.5 , a $1.85 \times$ higher k^* .

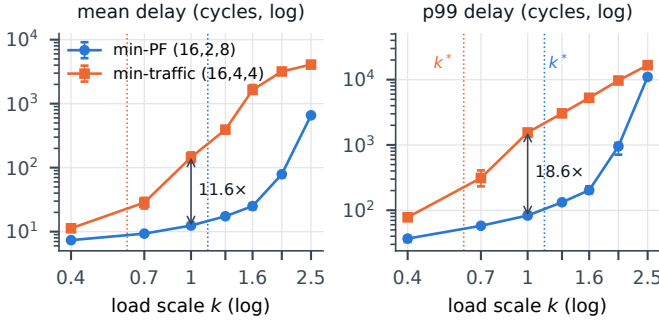


Fig. 2. Load response of ResNet-50 $c=16$ min-PF (2, 8) vs min-traffic (4, 4) under BL; bars span three placements, dotted lines mark each arm’s k^* .

At $k = 1.0$ the gap is $11.6\times$ in mean delay and $18.6\times$ in p99, and it widens with load. Both arms stay below their own floor throughout ($PL/PF \leq 0.58$), so the gap is not a placement artefact, and under DP the same rung reads $13.8\times / 20.3\times$.

The floor is therefore worth its bandwidth overhead, and the classical minimum-traffic objective is the wrong one to rank orientations by: packing sets the operating point that every downstream layer inherits.

E. Mapping Exploration

With the packing fixed, the mapping layer controls one quantity, PL, and we ask whether moving it moves delay. The population is ResNet-50 at its $c=8$ minimum-PF packing (2, 4), $PF = 0.392$: 24 placements produced by the guarded search, spanning $PL = 0.127\text{--}0.490$ ($3.9\times$) with path variety pinned so that PL is the only predictor that varies, seven of them above the floor. Each placement is simulated at two loads, $k = 1.60$ and 1.80 , with 30 seeds each, under local selection (BL).

Fig. 3 plots mean delay and p99 against PL at $k = 1.80$. Delay does not respond to PL until PL crosses PF and responds sharply afterwards: a regression with a hinge at the floor reaches $R^2 = 0.735$ against 0.404 for a straight line, with a below-floor slope indistinguishable from zero ($t = -0.16$) and an above-floor slope of $+655$ cycles per unit load ($t = +5.13$, $p < 10^{-4}$). The pattern replicates at $k = 1.60$ ($R^2 = 0.705$), with the above-floor slope $64\text{--}72\times$ the below-floor one at both loads. Delay therefore tracks $B = \max(PF, PL)$ ($r = +0.86$), and which of its two terms is active is decided by whether the placement has crossed the floor.

Hop count acts separately but weakly: on three arms of the same packing at identical B below the floor, 15% more communication cost adds 3.7–6.7% to free-flow delay, while adding CC to a B model moves its adjusted R^2 by at most 0.01 — communication cost sets where the delay curve starts, B where it bends.

Below the floor placement still matters in a band around the knee — eight placements of ResNet-50 (8, 1, 8) spread $1.34\times$, $13.62\times$ and $1.28\times$ in mean delay at light load, the knee and saturation — but searching past minimum-communication-cost placements returns at best $1.14\times$ ($1.34\times$ in p99) on about six

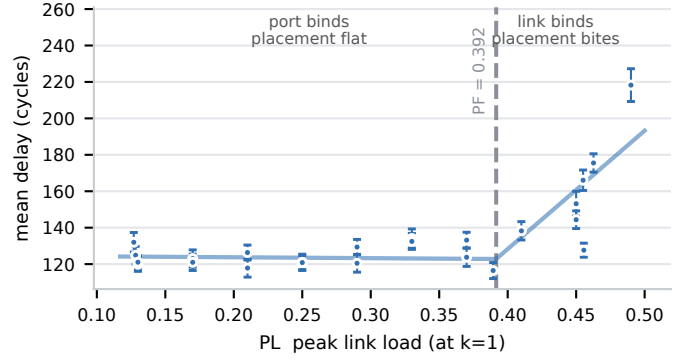


Fig. 3. Mean delay vs. PL, 24 placements of ResNet-50 (8, 2, 4) at $k = 1.80$ under BL, hinge fit at $PF = 0.392$.

pairs, so minimum communication cost already captures nearly all of it.

Above the floor the picture inverts. Across five above-floor populations on the three workloads, $n = 8$ to 24 placements each at three seeds and read in the knee window, delay spreads $4.6\text{--}31.8\times$ between the best and worst placement, and PL/PF orders that spread with a positive correlation in every population. Above the floor, placement determines delay and PL is what to search on.

Finally, five causes that could push a placement across the floor without intent (min-CC mapping, thermal spreading, link faults, TSV clustering, multi-tenancy) all leave $PL/PF \leq 1.000$ when screened offline: the above-floor regime is entered by provisioning, not by accident.

The mapping layer’s task below the floor is therefore to reach minimum communication cost and stay clear of the knee band. What a placement can still do for free once it is there is the subject of the next section.

F. Mapping Refinement

Below the floor the previous section leaves the mapping layer at minimum communication cost, and we ask whether anything is still free once communication cost and PL are held. Escapable load E does not predict capacity (k^* correlates with PL at -0.54 to -0.84 in five populations while E ’s sign varies), so escape room is tested for its effect on delay by intervention, since it co-varies with PL and communication cost in any sampled population. We therefore fix the packing and the seed, start from the minimum-communication-cost placement, and run two guarded climbs from it: one minimising and one maximising the escape slope $ES = E_{20} - E_5$, both constrained to $CC \leq CC_{\min}$ and $PL \leq PL(CC_{\min})$. The two arms then differ only in how escape room is distributed across the congested links. This is done on DeiT-S (16, 1, 16), VGG-16 (8, 4, 2) and ResNet-50 (8, 2, 4), eight seeds each, under both selection policies, read at each arm’s knee rung.

Table III reports the paired result. Maximising ES lowers mean delay by $1.13\text{--}1.25\times$ and p99 by $1.26\text{--}1.43\times$ at the arm level, and by up to $1.85\times$ and $2.99\times$ on the best individual placement — 46% of mean delay and 67% of p99 — at no cost in communication or peak link load. Combining the three

TABLE III
MAXES OVER MINES AT MATCHED CC AND PL, READ AT EACH ARM’S
KNEE RUNG ($n = 8$ SEEDS): ARM MEAN AND BEST PLACEMENT.

arm	rung	mean delay		p99	
		arm	best	arm	best
DeiT-S (16, 1, 16)	k 0.55, BL	1.25×	1.85×	1.43×	2.99×
VGG-16 (8, 4, 2)	k 0.55, BL	1.14×	1.64×	1.28×	2.25×
ResNet-50 (8, 2, 4)	k 1.15, DP	1.13×	1.32×	1.26×	1.69×

arms gives $p = 0.025$ on mean delay and $p = 0.016$ on p99 under BL, and $p = 0.046$ on mean delay under DP. The effect is confined to the knee (it is gone one rung past it) and it is not a property of the selection unit: which policy collects it varies by arm.

The lever has a boundary that is visible in advance: on VGG-16 (32, 4, 8), where one pair carries 92% of the peak link, the intervention returns nothing, and whether a guarded climb can move ES at all — an offline check costing minutes — called the outcome in all four cases, with all nine minimum-PF orientations on the responding side. Escape shape is therefore a zero-cost design objective for the mapping layer; what remains is decided at run time.

G. Routing and Selection

The last layer acts at run time. With packing and placement fixed, selection decides which of the admissible outputs a packet takes, and we compare the local buffer-level policy (BL) used so far against the distributed cost-to-go policy (DP) on identical placements and seeds, so that the policy is the only difference between paired runs.

On 71 above-floor placements from the populations of Sec. IV-E, read in the knee window, DP is faster than BL on 65, by $1.15\times$ in mean delay and $1.36\times$ in p99 on average and up to $1.64\times$ and $2.35\times$ on a single placement. Below the floor the same test gives a smaller gain that peaks at each packing’s own knee (1.10 – $1.35\times$ mean across three sets). The regimes differ in magnitude, not in sign.

DP’s second effect is on the *spread* of delay across placements, which Sec. IV-E showed to be the mapping layer’s main risk. We take every population in which BL exhibits such a spread — eight in all, spanning the three workloads, three densities, eight orientations and both regimes, $n = 7$ to 24 placements at three seeds — and read the ratio of the slowest to the fastest placement under each policy inside that population’s knee window. Fig. 4 shows one population in each regime.

DP narrows the spread in all eight populations, by $1.09\times$ to $7.50\times$. In Fig. 4 the equal-PL population above the floor moves from $64.3\times$ to $8.6\times$ in mean delay, and the knee population below it from $13.6\times$ to $7.6\times$ ($22.6\times$ to $14.0\times$ in p99). The mechanism is rescue of the worst placements rather than acceleration of the best: on the fastest placement of each population BL matches or beats DP, while on the slowest DP improves delay by up to $7.6\times$ (5 566 to 732 cycles). What DP cannot compress is variation that PL itself causes, since that

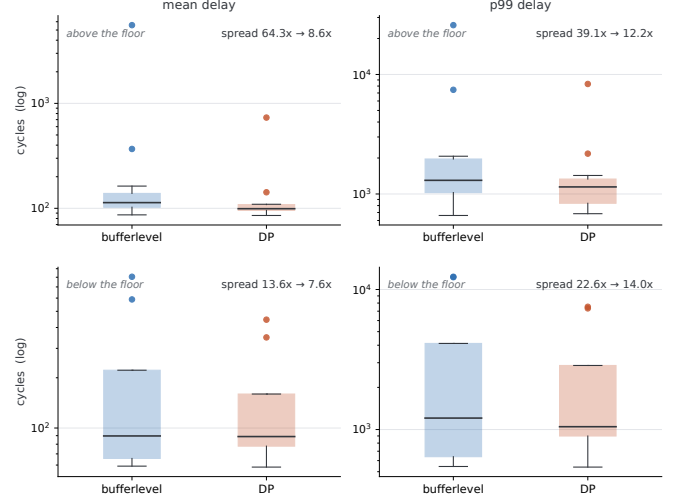


Fig. 4. Delay across placements, BL vs. DP: 12 equal-PL placements of ResNet-50 (16, 2, 8) above the floor (top) and 8 placements of (8, 1, 8) at the knee below it (bottom).

is B : where the spread comes from a PL range the ratio is unchanged.

The compression window is the knee window of Sec. IV-E: adaptive selection is a throughput lever above the floor and a robustness lever wherever placements differ, and both live at the knee.

H. Policy Choice and Its Limit

We ask which fixed policy is better over everything measured, and how much a policy that could switch per operating point would add. Every knee-window placement $\times$ load cell from the populations above is used — 701 above the floor, 128 below, three seeds each — and three policies are compared on identical cells: always-BL, always-DP, and a per-cell oracle taking the better of the two.

Fig. 5 shows the result. DP is ahead of BL in both regimes and at every load band: by $1.85\times$ in mean delay and $2.39\times$ in p99 above the floor, and by $1.15\times$ and $1.23\times$ below it. The oracle adds 1.1% and 2.5% above the floor but 3.8% and 7.7% below it: of the improvement available between always-BL and the oracle, always-DP captures 98% and 96% above the floor but only 78% and 71% below it.

The remaining 4–8% below the floor is the only headroom left, so we ask whether the cells DP loses can be identified at design time. About thirty offline quantities correlate with DP’s per-cell gain at $|r| \leq 0.4$, none surviving a held-out packing, and a pre-registered learned model over 15 offline features of 56 placements from seven packings reaches $R^2 = -0.03$ on DP’s delay and p99 gain when a packing is held out, against a registered ceiling of 0.10.

Adaptive selection is therefore the right fixed policy in both regimes, and what it leaves below the floor is not a design-time quantity.

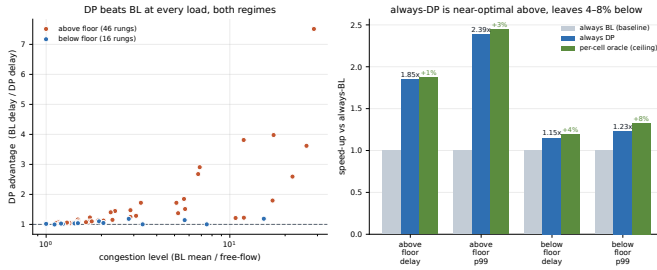


Fig. 5. Left: DP's advantage over BL per cell vs. congestion. Right: speed-up over always-BL of always-DP and of a per-cell oracle, by regime.

Scope. PL, PL_f and E are offline path-enumeration models under an even-spreading assumption that BL, in particular, does not meet. PL has been checked against the simulator on one placement (seven links spanning ranks 1–50: measured/model 0.88–1.01 under BL, rank order $\rho = +0.89$); per-flow path counts have not, and none is quoted. All results use one $6 \times 6 \times 3$ mesh and one buffer depth, neither swept. Partial sums are carried at INT32, which overstates the reduction traffic the flow exploits; placement rankings survive at 16-bit on ResNet-50 and VGG-16. Spread and best-case ratios are max/min statistics over 7–24 placements at three seeds.

V. CONCLUSION

This paper presents a cross-layer design flow for IMC DNN accelerators on a 3D NoC. Prior studies design for a single layer, which limits design freedom and forgoes the gains shown here. The proposed flow is evaluated on three workloads. The design layers are found to be highly coupled. The first layer (packing), which decides the port floor, is shown to have the highest impact: the optimal design choice for delay lowers mean delay $11.6\times$ and p99 $18.7\times$ at matched load. Below that floor, mapping can be optimized through a proposed metric (escape slope); around the knee, at no communication cost, this metric is shown to recover up to 46% of mean delay and 67% of p99. At the selection layer, adaptive selection is the better fixed policy in both regimes, by $1.85\times/2.39\times$ above the floor and $1.15\times/1.23\times$ below it.

Always-DP is shown to capture 96–98% of the improvement an oracle switching policy per placement and load would achieve above the floor and 71–78% below it. No offline quantity predicts which cells it loses, because the residue is temporal: operating below the floor produces transient congestion clustered at phase boundaries, on a timescale a spatially adaptive policy cannot track. Closing this residue motivates a temporally as well as spatially adaptive selection policy. The same design flow admits other objectives, such as thermal-aware placement and load balancing.

REFERENCES

- [1] N. P. Jouppi *et al.*, “In-Datacenter Performance Analysis of a Tensor Processing Unit,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, vol. 45. New York, NY, USA: Association for Computing Machinery, Jun. 2017, pp. 1–12.
- [2] M. Horowitz, “1.1 Computing’s energy problem (and what we can do about it),” in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 2014, pp. 10–14.

- [3] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, “Efficient Processing of Deep Neural Networks: A Tutorial and Survey,” *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [4] A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh, and E. Eleftheriou, “Memory devices and applications for in-memory computing,” *Nature nanotechnology*, vol. 15, no. 7, pp. 529–544, 2020.
- [5] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramanian, J. P. Strachan, M. Hu, R. S. Williams, and V. Srikumar, “ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars,” in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 14–26.
- [6] P. Chi, S. Li, C. Xu, T. Zhang, J. Zhao, Y. Liu, Y. Wang, and Y. Xie, “PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory,” in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 27–39.
- [7] Y.-H. Chen, J. Emer, and V. Sze, “Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks,” in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 367–379.
- [8] Y. Long, D. Kim, E. Lee, P. Saha, B. A. Mudassar, X. She, A. I. Khan, and S. Mukhopadhyay, “A Ferroelectric FET-Based Processing-in-Memory Architecture for DNN Acceleration,” *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits*, vol. 5, no. 2, pp. 113–122, 2019.
- [9] S. M. Nabavinejad, M. Baharloo, K.-C. Chen, M. Palesi, T. Kogel, and M. Ebrahimi, “An Overview of Efficient Interconnection Networks for Deep Neural Network Accelerators,” *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 10, no. 3, pp. 268–282, Sep. 2020.
- [10] G. Krishnan, S. K. Mandal, C. Chakrabarti, J.-s. Seo, U. Y. Ogras, and Y. Cao, “Impact of On-chip Interconnect on In-memory Acceleration of Deep Neural Networks,” *ACM Journal on Emerging Technologies in Computing Systems*, vol. 18, no. 2, Apr. 2022.
- [11] G. Krishnan, S. K. Mandal, M. Pannala, C. Chakrabarti, J.-S. Seo, U. Y. Ogras, and Y. Cao, “SIAM: Chiplet-based Scalable In-Memory Acceleration with Mesh for Deep Neural Networks,” *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 20, no. 5s, pp. 68:1–68:24, Sep. 2021.
- [12] S. K. Mandal, G. Krishnan, C. Chakrabarti, J.-S. Seo, Y. Cao, and U. Y. Ogras, “A Latency-Optimized Reconfigurable NoC for In-Memory Acceleration of DNNs,” *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 10, no. 3, pp. 362–375, Sep. 2020.
- [13] X. Li, W. Fan, H. Zhang, J. Ji, T. Cheng, S. Li, L. Li, and Y. Fu, “TTNNM: Thermal- and Traffic-Aware Neural Network Mapping on 3D-NoC-based Accelerator,” in *Proceedings of the Great Lakes Symposium on VLSI 2024*, ser. GLSVLSI ’24. Clearwater, FL, USA: Association for Computing Machinery, 2024, pp. 364–369.
- [14] G. Narang, C. Ogbogu, J. R. Doppa, and P. P. Pande, “TEFLON: Thermally Efficient Dataflow-aware 3D NoC for Accelerating CNN Inference on Manycore PIM Architectures,” *ACM Trans. Embed. Comput. Syst.*, vol. 23, no. 5, Aug. 2024.
- [15] N. Dahir, T. Mak, F. Xia, and A. Yakovlev, “Highly Adaptive and Deadlock-Free Routing for Three-Dimensional Networks-on-Chip,” *IET Computers & Digital Techniques*, vol. 7, no. 6, pp. 255–263, 2013.
- [16] T. Mak, P. Y. Cheung, W. Luk, and K. P. Lam, “A DP-network for optimal dynamic routing in network-on-chip,” in *Proceedings of the 7th IEEE/ACM International Conference on Hardware/Software Codesign and System Synthesis*, ser. CODES+ISSS ’09. Grenoble, France: Association for Computing Machinery, 2009, pp. 119–128.
- [17] W. Choi, K. Duraisamy, R. G. Kim, J. R. Doppa, P. P. Pande, D. Marculescu, and R. Marculescu, “On-Chip Communication Network for Efficient Training of Deep Convolutional Networks on Heterogeneous Manycore Systems,” *IEEE Transactions on Computers*, vol. 67, no. 5, pp. 672–686, May 2018.
- [18] T. Miyajima and L. Fukuoka, “Benchmarking the Cerebras Wafer Scale Engine-2 Architecture,” in *Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis (SC-W)*, ser. SC Workshops ’25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 818–822.
- [19] V. Catania, A. Mineo, S. Monteleone, M. Palesi, and D. Patti, “Cycle-Accurate Network on Chip Simulation with Noxim,” *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, vol. 27, no. 1, pp. 4:1–4:25, Aug. 2016.

- [20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2016, pp. 770–778.
- [21] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *CoRR*, Sep. 2014.
- [22] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *International Conference on Machine Learning*, Dec. 2020.